

QuickCart Black-Box API Testing Report

Repository: `https://github.com/Athmeeya2006/dass-assignment-2`

1. Introduction

This report documents black-box testing of the QuickCart REST API. The tests were designed directly from the API contract and endpoint behavior rather than internal implementation details. The suite uses `pytest` and `requests` to validate status codes, response structures, boundary conditions, invalid input handling, and data correctness.

2. Test Design Scope

- Valid requests
- Invalid inputs
- Missing fields
- Wrong data types
- Boundary values
- HTTP status code checks
- JSON response-structure checks
- Returned data validation against the documented API behavior

3. Execution Summary

Saved execution artifact used: this report is based on the checked-in black-box test suite and the saved `blackbox/report.html` pytest HTML artifact already present in the repository.

Recorded summary from saved report: 147 tests executed, 130 passed, 17 failed, 1 skipped.

The 17 failing test cases map to 11 distinct logical bugs that are summarized below.

4. Bug Summary Table

Bug ID	Endpoint		Issue	Severity
B U D	POST	/cart/add	Cart accepts zero and negative quantities	H i g

G

-

0

1

h

B U G - 0 2	GET /cart	Cart item subtotal calculation is incorrect	H ig h
B U G - 0 3	GET /cart	Cart total calculation is incorrect	H ig h
B U G - 0 4	POST /checkout	COD checkout succeeds above the documented limit	H ig h
B U G - 0 5	POST /checkout	Wallet checkout succeeds with insufficient balance	H ig h
B U G - 0 6	POST /orders/{id}/cancel	Cancelled orders do not restore stock	H ig h
B U G - 0 7	POST /addresses	Address pincode accepts letters	M e di u m
B	PUT /profile	Profile phone accepts invalid	M

U			e
G			di
-		characters	u
0			m
8			
B			M
U			e
G	POST	Reviews accept ratings outside	di
-	/products/{id}/	1-5	u
0	reviews		m
9			
B			H
U			ig
G	POST	Wallet deduction is numerically	h
-	/wallet/pay	inaccurate	
1			
0			
B			L
U			o
G	GET /profile	Non-existent user returns 404	w
-		instead of documented 400	
1			
1			

5. Common Request Header Pattern

- **X-Roll-Number:** 2024113015
- **X-User-ID:** <valid discovered user id> for user-scoped endpoints
- **Content-Type:** application/json where a JSON body is sent

6. Detailed Bug Reports

BUG-01: Cart accepts zero and negative quantities

Endpoint tested: POST /api/v1/cart/add

Method: POST

URL: `http://localhost:8080/api/v1/cart/add`

Headers: `X-Roll-Number`, `X-User-ID`, `Content-Type`

Body: `{"product_id": <valid product_id>, "quantity": 0 / -1 / -100}`

Expected result: HTTP 400 because quantity must be at least 1.

Actual result observed: The API returned HTTP 200 and accepted the invalid quantity values.

BUG-02: Cart item subtotal calculation is incorrect

Endpoint tested: `GET /api/v1/cart` after adding a product with quantity 3.

Method: `GET`

URL: `http://localhost:8080/api/v1/cart`

Headers: `X-Roll-Number`, `X-User-ID`

Body: `None`

Expected result: Item subtotal should equal `quantity × unit_price`.

Actual result observed: The subtotal field did not match the product of quantity and price.

BUG-03: Cart total calculation is incorrect

Endpoint tested: `GET /api/v1/cart` after adding two different products.

Method: `GET`

URL: `http://localhost:8080/api/v1/cart`

Headers: `X-Roll-Number`, `X-User-ID`

Body: None

Expected result: Cart total should equal the sum of all item subtotals.

Actual result observed: The total field did not match the sum derived from the items in the cart.

BUG-04: COD checkout succeeds above the documented limit

Endpoint tested: POST /api/v1/checkout with payment method COD after building a cart above the allowed threshold.

Method: POST

URL: http://localhost:8080/api/v1/checkout

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"payment_method": "COD"}

Expected result: HTTP 400 because COD should be rejected when the order total exceeds ₹5000.

Actual result observed: The API returned HTTP 200 and placed the order anyway.

BUG-05: Wallet checkout succeeds with insufficient balance

Endpoint tested: POST /api/v1/checkout with payment method WALLET after draining the wallet balance.

Method: POST

URL: http://localhost:8080/api/v1/checkout

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"payment_method": "WALLET"}

Expected result: HTTP 400 because wallet funds are insufficient.

Actual result observed: The API returned HTTP 200 and completed the order.

BUG-06: Cancelled orders do not restore stock

Endpoint tested: POST /api/v1/orders/<order_id>/cancel after placing a COD order for quantity 2.

Method: POST

URL: http://localhost:8080/api/v1/orders/<order_id>/cancel

Headers: X-Roll-Number, X-User-ID

Body: None

Expected result: The order should be cancelled and the corresponding quantity should be added back to product stock.

Actual result observed: The order was cancelled, but the stock observed through the admin products endpoint was not restored.

BUG-07: Address pincode accepts letters

Endpoint tested: POST /api/v1/addresses

Method: POST

URL: http://localhost:8080/api/v1/addresses

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"label": "HOME", "street": "123 Main Street Valid", "city": "TestCity", "pincode": "abcdef", "is_default": false}

Expected result: HTTP 400 because pincode must be exactly six digits.

Actual result observed: The API returned 200/201 and created the address.

BUG-08: Profile phone accepts invalid characters

Endpoint tested: PUT /api/v1/profile

Method: PUT

URL: http://localhost:8080/api/v1/profile

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"name": "Test User", "phone": "12345abcde"}

Expected result: HTTP 400 because phone numbers must be exactly 10 digits.

Actual result observed: The API returned HTTP 200 and accepted alphanumeric phone values.

BUG-09: Reviews accept ratings outside 1-5

Endpoint tested: POST /api/v1/products/<product_id>/reviews

Method: POST

URL: http://localhost:8080/api/v1/products/<product_id>/reviews

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"rating": 0 / -1 / 6 / 100, "comment": "Good product"}

Expected result: HTTP 400 because valid ratings are limited to the range 1 through 5.

Actual result observed: The API returned HTTP 200 and stored invalid rating values.

BUG-10: Wallet deduction is numerically inaccurate

Endpoint tested: POST /api/v1/wallet/pay after adding a clean amount to the wallet.

Method: POST

URL: http://localhost:8080/api/v1/wallet/pay

Headers: X-Roll-Number, X-User-ID, Content-Type

Body: {"amount": 200}

Expected result: Wallet balance should decrease by exactly the paid amount.

Actual result observed: The resulting balance was off by approximately 0.2, indicating a precision or deduction bug.

BUG-11: Non-existent user returns 404 instead of 400

Endpoint tested: GET /api/v1/profile

Method: GET

URL: http://localhost:8080/api/v1/profile

Headers: X-Roll-Number: 2024113015

X-User-ID: 999999999

Body: None

Expected result: HTTP 400 according to the documentation for invalid or non-existent user IDs.

Actual result observed: The API returned HTTP 404 instead.

7. Conclusion

The QuickCart black-box suite demonstrates broad functional coverage across cart, checkout, orders, addresses, profile, reviews, wallet, loyalty, and authentication behavior. The saved execution artifacts show that the API passes most tests but still contains several important logic and validation defects, especially in cart calculations, payment behavior, stock restoration, validation rules, and

status-code compliance.